

Distributed AI: motion tubes explained

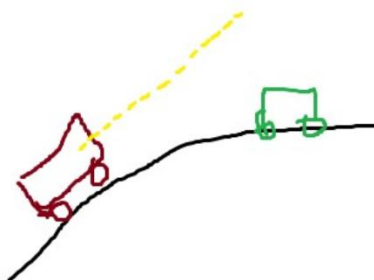
Core Idea

We are not doing a real visually “lane detection”, but a kinematic prediction of the lane we are currently driving in. We use the **steering angle** and the speed of the vehicle and we assume we have a flat road and project the left and right lane edges onto the camera picture. As a result we get:



Advantages: we do not use labels or a neural network, it's pretty light and stable.

Disadvantages: if the road bends or splits while you are still steering straight ahead, the tube does not “see” this. For example:



Here we see the road is not flat (there is a hill), therefore the red car will not see the green car.



We see here to our left, a hill where the road is going down and the lanes are drawn ‘straight up in the air’ and doesn’t detect the right curve.

In short: we do forward projection. We build the lane in 3D (on the flat road surface) from the car's kinematics, convert those 3D points directly to camera coordinates and project it to pixels using the pinhole model (`cv2.projectPoints`).

Understanding the setup

First, we take every simulation step the vehicle dynamics out of CARLA. In `run_world.py` we read the control input that the autopilot just used: `ctrl.steer` $\in [-1,1]$. This is a normalised steer position; we are scaling this to a real steer angle in radians with following code line:

```
steer_rad = -float(ctrl.steer) * MAX_STEER_RAD      (MAX_STEER_RAD is chosen)
```

The minus sign corrects the direction so that “steer right” actually becomes a right turn in our camera coordinate system. Otherwise, if the vehicle goes left then our lane detection goes right.

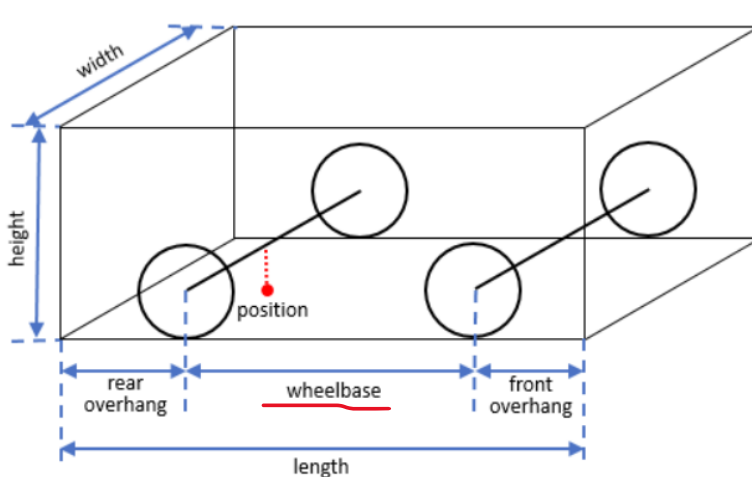
Next, we get the speed out of the speedvector of the vehicle and we calculate the standard (‘de norm’ in Dutch) of it and that’s the speed “speed_ms” in m/s. Both speed_ms and steer_rad are written in a shared memory (`VEHICLE_STATE_MEMORY.dat`) and then read in visualisation process (`run_vehicle_pov.py`) so we can use it.

So basically, first we run `run_world.py` where we simulate all the traffic and write the status of the vehicle to a memory storage and then after running `run_world.py` we read the information that was written to that same storage memory. In this way, we can get the status.

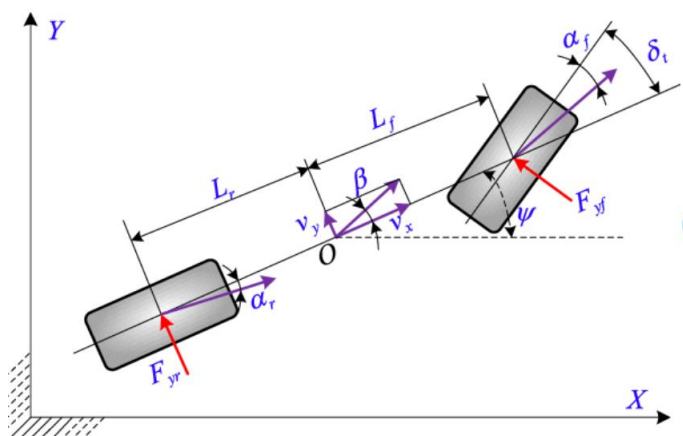
‘Motion tubes’ mathematical and technical overview

In the code `motion_tubes.py` (`app/data_processors/motion_tubes.py`) you’ll find the class `MotionTubeProjector`. In this class we convert that steering angle into a curve curvature using the standard bicycle model (Ackermann approximation).

First, let’s explain the word ‘**wheelbase**’: The wheelbase is the distance (in meters) between the center of the rear axle and the center of the front axle. It’s a fixed vehicle parameter: for a Tesla Model 3, it’s around 2,875 m; in our code, we use $L \approx 2.8$. This length appears in all vehicle kinematic models because it determines how much the vehicle turns for a given steering angle.



Now those 4 wheels can be replaced by 2 "collapsed" wheels for easier math calculations, one at the front axle and one at the back axle. This is called the bicycle model. We then get 2 wheels (ignore all those labels, just know we have 2 wheels that look like this instead of 4):

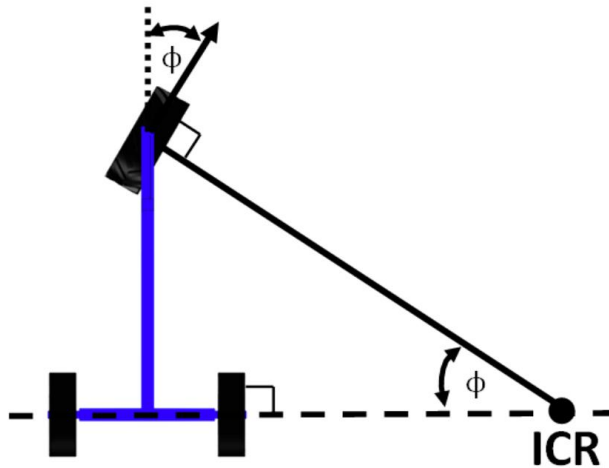


We can do this if we assume the following:

- The tyres do not slip
- The front wheel set steers at an angle δ relative to the vehicle's axle; the rear wheels are straight ahead. This illustrates the picture above.
- Motion is flat (ignoring roll and pitch and elevation changes in the road).
- We consider a short time interval in which δ and v (velocity) are constant.

(a little bit ICR now but not very important, just know it exists)

Under these assumptions, there exists an **instantaneous center of rotation (ICR)**: the center of the arc around which the car is rotating at that instant. Any velocity in the vehicle is then a rotation with angular velocity r around this ICR point. This ICR point is not a part of the vehicle itself, it's just a point virtual that exists.



(in this picture 3 tyres but we have now reduced to 2).

For a rigid body (Dutch: star lichaam) moving in a plane without slip, there exists at any instant a point in the plane about which the body rotates. This point is called the instantaneous (meaning "currently valid") center of rotation. Don't get too much into detail here since it's kinematics.

So now that we know how the wheelbase L and angle δ are defined. We get the following formula to calculate the curve (k) of the path, this is called the inverse Ackermann-relation, unit of k is $1/m$ (since we have radians which are unitless divided by m):

$$k = \tan(\delta)/L$$

- L = wheelbase in m (it's a constant of the car)
- δ = angle in radians

The bigger δ , the greater curvature. In CARLA is $\text{steer} > 0$ a bend to the right, but because of our – (see also page 2) then going left is now positive. So this is pure a formula for a prediction for where my vehicle is going (no lane markers/detections up till here).

Definition of curvature = change in orientation per arc length traveled:

$$\kappa = \frac{d\psi}{ds}$$

So now based on that curvature we would like to construct the path in the plane of the vehicle. It is most convenient to parameterize not in time but arc length s (booglengthe, meters along the track). Start in $s = 0$ at the rear axle center and let the vehicle direction there be the x -axis. For a flat surface:

- the orientation $\theta(s)$ changes with $d\theta/ds = \kappa$ (because curvature = change of direction per meter)

- and the position evolves according to $dx/ds=\cos\theta(s)$, $dy/ds=\sin\theta(s)$

Since k is constant within our short window, this integrates exactly:

$$\theta(s) = \kappa s,$$

$$x(s) = \int_0^s \cos(\kappa\sigma) d\sigma = \frac{\sin(\kappa s)}{\kappa}, \quad y(s) = \int_0^s \sin(\kappa\sigma) d\sigma = \frac{1 - \cos(\kappa s)}{\kappa}.$$

So these formulas describe the curvature, math behind less important, just know we can use this to calculate the following:

- $x(s)$ = how much m forwards
- $y(s)$ = how much to the left

If $k = 0$ then $x(s)$ is equal to s and $y(s)$ is equal to 0. Basically a straight line.

All this is very abstract and can be difficult. So let's go now more practical.

More practical

In practice, the first step is to decide how far ahead we want to predict the ego vehicle's path. In our code we compute a horizon distance in meters using the current speed:

```
horizon_m = clip(speed_ms*2.0, 30.0, meters_ahead)
```

This means we always predict at least 30 meters ahead, and at most `meters_ahead` (we set this to 40 meters). Over this interval we sample a set of points s from 0 to `horizon_m`. Here, s is the distance along the predicted path (arc length), not the curvature. We use roughly 4 samples per meter so the curve is smooth enough for drawing and for later intersection checks.

For each sampled value s , we compute the center trajectory using a simple bicycle model. First we compute the curvature k from the steering angle δ and the wheelbase L :

$$k = \tan(\delta)/L$$

In the actual implementation the steering angle is also slightly smoothed with an exponential moving average before it is used, to reduce jitter and make the tube more stable from frame to frame.

If k is not close to zero (so no straight line), the predicted centerline is an arc and we compute: $x(s) = \sin(ks)/k$ and $y(s) = (1 - \cos(ks))/k$.

Otherwise, if $k \approx 0$, the trajectory becomes a straight line and we use the approximation: $x(s) = s$ and $y(s) = 0$.

All points $(x(s), y(s))$ are stored in an array `center_xy`, expressed in the vehicle coordinate frame and in meters. Each row $[x,y]$ can be interpreted as: “this point lies x meters in front of the vehicle reference point and y meters to the left.” At this stage, the curve is still only defined in vehicle coordinates. This is not yet in camera view.

From vehicle coordinates to camera image

Up to this point, we have constructed a trajectory in the vehicle coordinate frame. For each arc length s , a point $(x(s), y(s))$ is computed in meters, where x represents how far the vehicle moves forward and y represents how far it moves to the left. This curve describes the predicted path of the ego vehicle based on its current speed and steering angle. However, this trajectory only exists in a mathematical plane attached to the vehicle. In order to visualise it in the camera image, several additional steps are required.

First, the single center trajectory must be expanded into a lane tube by generating left and right lane boundaries. Next, these 2D ground points must be lifted into 3D camera coordinates using a simple geometric camera model. Finally, the 3D points are projected onto the image plane using a pinhole projection model so they can be drawn as pixels.

Building the lane tube in the vehicle frame

The construction of the trajectory starts in the following function: `MotionTubeProjector._centerline_xy`. This function returns an array called `center_xy` with shape $(N,2)$, where each row contains a point $[x,y]$ expressed in meters in the vehicle frame. This centerline represents the predicted path of the ego vehicle along the road.

To obtain a lane tube instead of a single line, the centerline is shifted sideways by half of the lane width. Since the y -axis in the vehicle frame points to the left, adding a positive offset moves the line to the left and subtracting the same offset moves it to the right. Using this idea, three trajectories are defined: the left lane boundary, the lane center, and the right lane boundary. Mathematically, these correspond to $(x(s), y(s)+w/2)$, $(x(s), y(s))$ and $(x(s), y(s)-w/2)$, where w is the lane width in meters.

In the code, this shifting is handled inside the function `_veh_to_cam_points` through the parameter `side_offset`. A positive value produces the left boundary, zero produces the centerline, and a negative value produces the right boundary. At this stage, all three curves still exist purely in the vehicle coordinate system and are defined in meters.

Converting from vehicle frame to camera frame

The next step is to express these points in the camera coordinate system that OpenCV uses for projection. To keep the system simple and efficient, a fixed and idealised camera model is assumed. The camera is rigidly mounted in the center of the vehicle at a constant height above the road. Its optical axis points straight forward, and the road surface is assumed to be flat.

In the vehicle frame, the x-axis points forward and the y-axis points to the left. In contrast, the OpenCV camera frame is defined with the X-axis pointing to the right, the Y-axis pointing downward in the image, and the Z-axis pointing forward. To convert between these frames, the forward distance remains unchanged and becomes the camera's forward axis. The lateral position is negated so that left in the vehicle frame corresponds to negative right in the camera frame. The vertical coordinate is fixed to the camera height, since all points are assumed to lie on the ground plane below the camera.

As a result, each 2D ground point is transformed into a 3D camera point (X,Y,Z), where X points right, Y points downward by the camera height, and Z points forward. Points that are too close to the camera or behind it are discarded to avoid unstable projections.

This step directly encodes the flat-world assumption. Because all points are forced onto a horizontal plane at a fixed height below the camera, the motion tube can appear to float above the road or sink into it when driving on slopes or hills. The real road geometry changes, but the model assumes it does not.

Pinhole projection to image pixels

Once the lane tube points are expressed in the camera coordinate frame, they can be projected onto the image plane. This is done using a standard pinhole camera model. In the constructor of the MotionTubeProjector, a simple intrinsic camera matrix is built from the image width, image height, and horizontal field of view. The focal length is computed from the field of view, and the principal point is assumed to lie in the center of the image. Lens distortion is ignored.

Using this intrinsic matrix, OpenCV's `cv2.projectPoints` function projects each 3D camera point (X,Y,Z) to a 2D pixel coordinate (u,v). Points that fall outside the image boundaries are removed. The remaining pixel coordinates are then rounded to integer values and reshaped into the polyline format expected by OpenCV.

The function `get_projected_lanes` applies this entire process to the left, center, and right lane curves and returns three polylines in image coordinates. These polylines are exactly what is later passed to the visualisation module to be drawn on top of the camera image.

Temporal smoothing of the trajectory

Something I may not mention very explicitly but also important is that before computing the curvature of the path, the steering angle is passed through an exponential moving average filter. This smoothing step reduces the effect of small, rapid fluctuations in the steering signal, which are common when the autopilot performs continuous corrections. With a smoothing factor of approximately 0.85, the resulting trajectory reacts slightly slower than the raw steering input but appears much more stable. This significantly reduces visual jitter and flickering of the motion tube from frame to frame.

Interaction with object detection and shared memory

The motion tube is not only used for visualisation, but also plays an important role in understanding the driving scene and communicating information between processes. The current vehicle speed and steering angle are written each simulation tick to a shared memory block called `VehicleStateMemory`. The visualisation and perception process reads these values and uses them as input for the motion tube computation.

At the same time, objects such as vehicles, pedestrians, and traffic lights are detected using a YOLO-based object detector. For each detected object, the `IntersectionDetector` computes the minimum distance between the center of the object's bounding box and the centerline of the projected motion tube. If this distance is smaller than a predefined threshold, the object is considered to lie inside the ego vehicle's lane. This information is then used by the visualisation module, which draws objects inside the tube in red and objects outside the tube in green.

Finally, the geometry of the lane tube can be shared with other components, such as a reinforcement learning agent. The left and right lane boundaries are normalised with respect to the image size and stored in `LaneTubeMemory`, a shared memory structure that can be read by the agent. This allows the agent to reason about lane geometry without having to rerun any computer vision or projection code itself.